به نام خدا

رضا آقانظری / 403522281 / گزارش تمرینات عملی / دکتر محمدی

1 – شرح منطق پیاده سازی عملگرهای همسایگی و چگونگی تضمین بدون دور ماندن مسیر ها :

منطق پیاده سازی همسایه ها : عملگر تولید همسایه در این سوال get_neighbors به گونه ای طراحی شده از چند همسایه فعلی به همسایه جدید می رسیم که 4 نوع تغییر داریم :

```
neighbor = []
t_step = problem.t_step

for delta in [-t_step, t_step]:
    new_time = state.departure_time + delta
    if problem.t_min <= new_time <= problem.t_max:
        new_state = State(path=state.path, departure_time=new_time)
        if problem.is_valid_state(new_state):
            neighbor.append(new_state)
```

1. تغییر زمان حرکت : زمان حرکت به اندازه یک قدم t_step به جلو یا عقب می رود و حتما چک می شود که داخل بازه بین t_min تا t_max باشد .
2. تغییر مسیر حرکت : ابتدا گره های میانی به صورت shuffle مرتب می شوند ، سپس برای هر گره میانی تلاش می شود با یک قسمتی از مسیر عوض شود . برای پیدا کردن مسیرهای جایگزین از BFS استفاده می کنیم که محدودیت هایی نیز دارد : یک:طول مسیر جایگزین حداکثر 8 گره باشد(علت این محدودیت، جلوگیری از پیچیده سازی مسئله و پرهیز از بررسی کل گراف) ،دو : گره های دیگر مسیر فعلی ممنوع هستند ، سه : مسیر هایی پذیرفته می شود که از نظر ساختاری معتبر باشند(is_valid_state)
3. ترکیب زمان و مسیر : اعمال می شود تا همسایه جدیدتر پیدا کنیم.
4. حذف تکراری ها

چگونگی تضمین بدون دور ماندن مسیر ها : دو مکانیزم پیاده سازی کردیم : یک : ایجاد یک مجموعه visited تا از بازگشت مجدد به همان گره در طول جستجو جلوگیری شود. دو : همچنین قبل از پذیرش نهایی به is_valid_state مسیر تولید شده را می دهیم ، خود این تابع به is_valid_path پاس می دهد و با بررسی برابری طول مسیر با تعداد گره یکتا ، وجود گره های تکراری را تشخیص می دهد و دور ها را رد می کند .

```
def is_valid_path(self, path: tuple) -> bool:
    """
    Validates that a path is non-empty, loop-free, starts at origin,
    ends at destination, and consists solely of existing network edges.
    """
    if not path or path[0] != self.origin or path[-1] != self.destination:
        return False
    if len(path) != len(set(path)):
        return False  # Contains cycles
    for i in range(len(path) - 1):
        if not self.network.has_edge(path[i], path[i + 1]):
            return False
    return True
```

2 – جدول مقایسه ای عملکرد هر 4 الگوریتم :

ابتدا برای اجرا دستورات طبق Readme.md باید حالت های زیر را داشته باشیم :

- `rush_hour.json & toy_graph.json`

- `dual_peak_rush_hour.json & metropolis_graph.json`
- `cascading_gridlock.json & metropolis_graph.json`
- `crossover_showcase.json & metropolis_graph.json`
- `flat_plateau.json & downtown_grid.json`

به صورت جداگانه اجرا کردیم ، سپس برای تمیزی و زیبایی داخل پایتون جدول با همان دیتاها رسم شد(Matplotlib , pyplot):

- `rush_hour.json & toy_graph.json`

```
BENCHMARK SUMMARY
======================================================================
* Hill Climbing          :  41.77 min | Depart: 08:45 | Path: Home -> B1 -> B2 -> H2 -> Uni
* Simulated Annealing    :  48.29 min | Depart: 08:45 | Path: Home -> H1 -> B1 -> B2 -> Uni
* Local Beam Search      :  41.77 min | Depart: 08:45 | Path: Home -> B1 -> B2 -> H2 -> Uni
* Genetic Algorithm      :  45.01 min | Depart: 07:45 | Path: Home -> B1 -> B2 -> Uni
======================================================================
```

- `dual_peak_rush_hour.json & metropolis_graph.json`

```
BENCHMARK SUMMARY
======================================================================
* Hill Climbing          :  31.69 min | Depart: 08:45 | Path: Home -> C1 -> C2 -> T2 -> Uni
* Simulated Annealing    :  38.57 min | Depart: 08:45 | Path: Home -> N1 -> B1 -> C2 -> T2 -> Uni
* Local Beam Search      :  31.69 min | Depart: 08:45 | Path: Home -> C1 -> C2 -> T2 -> Uni
* Genetic Algorithm      :  31.69 min | Depart: 08:45 | Path: Home -> C1 -> C2 -> T2 -> Uni
======================================================================
```

- `cascading_gridlock.json & metropolis_graph.json`

```
BENCHMARK SUMMARY
======================================================================
* Hill Climbing          :  31.02 min | Depart: 08:45 | Path: Home -> C1 -> C2 -> T2 -> Uni
* Simulated Annealing    :  32.00 min | Depart: 08:45 | Path: Home -> N1 -> B1 -> C2 -> T2 -> Uni
* Local Beam Search      :  31.02 min | Depart: 08:45 | Path: Home -> C1 -> C2 -> T2 -> Uni
* Genetic Algorithm      :  31.02 min | Depart: 08:45 | Path: Home -> C1 -> C2 -> T2 -> Uni
======================================================================
```

- `crossover_showcase.json & metropolis_graph.json`

```
BENCHMARK SUMMARY
======================================================================
* Hill Climbing          :  32.00 min | Depart: 07:45 | Path: Home -> N1 -> B1 -> C2 -> B2 -> Uni
* Simulated Annealing    :  32.00 min | Depart: 07:45 | Path: Home -> N1 -> B1 -> C2 -> B2 -> Uni
* Local Beam Search      :  31.01 min | Depart: 08:45 | Path: Home -> C1 -> C2 -> B2 -> Uni
* Genetic Algorithm      :  32.00 min | Depart: 07:45 | Path: Home -> N1 -> B1 -> C2 -> B2 -> Uni
======================================================================
```

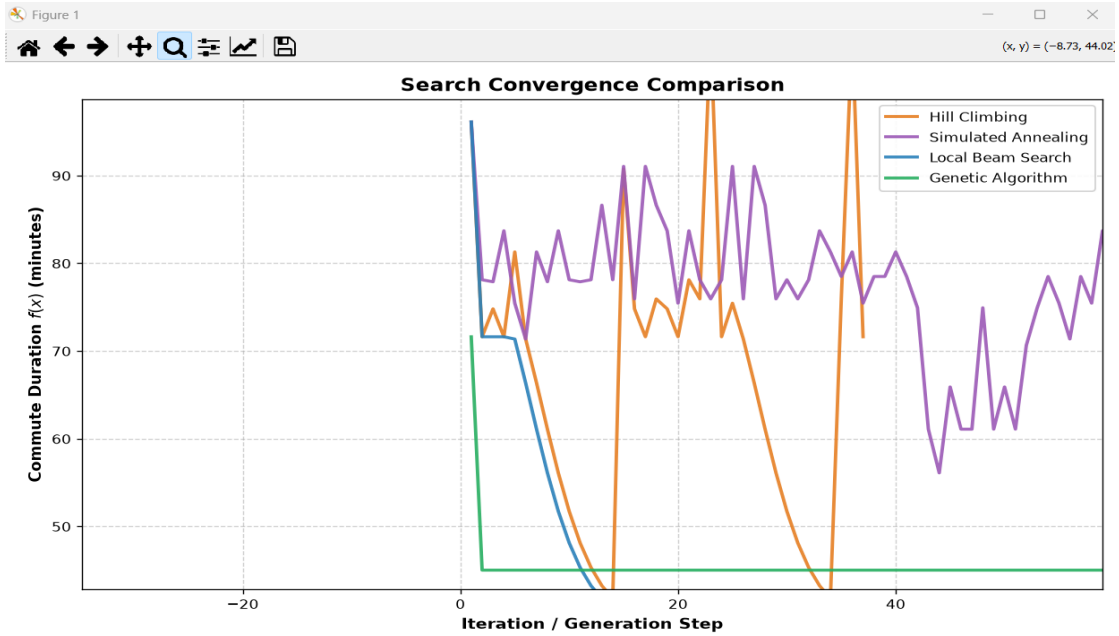
flat_plateau.json & downtown_grid.json

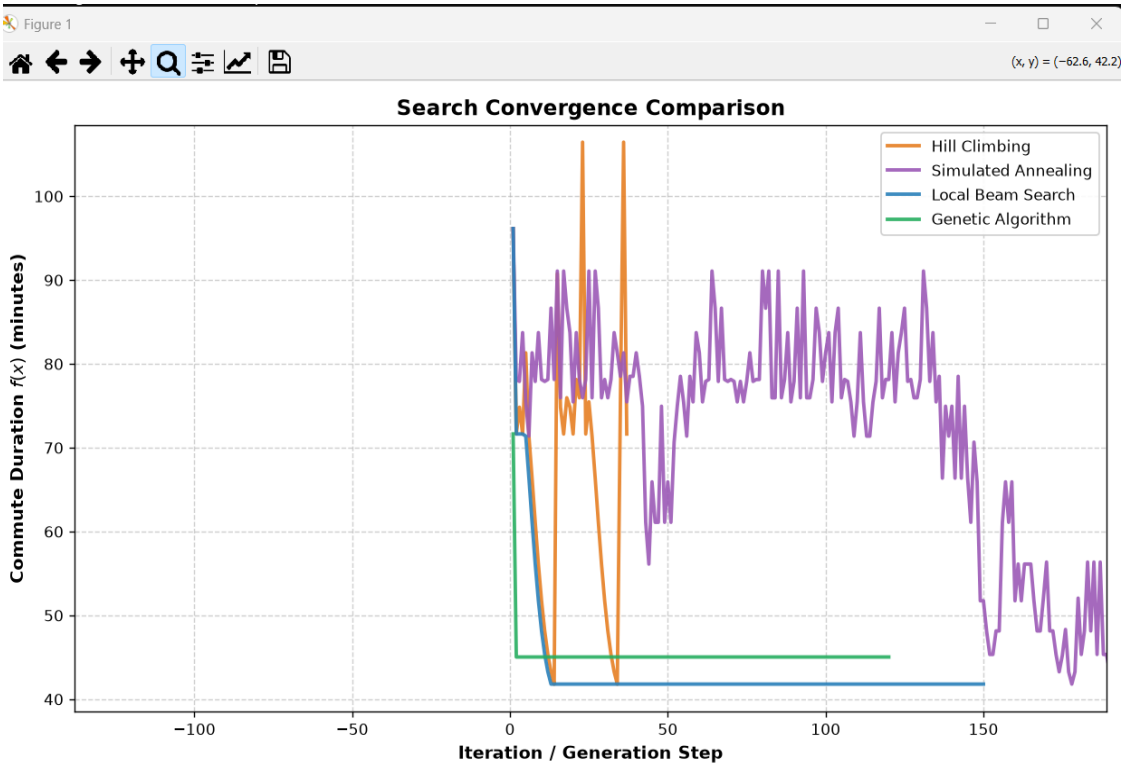
```
BENCHMARK SUMMARY
================================================================
* Hill Climbing          :  28.85 min | Depart: 08:45 | Path: Home -> G1 -> G2 -> G3 -> G7 -> Uni
* Simulated Annealing    :  28.85 min | Depart: 08:45 | Path: Home -> G1 -> G2 -> G3 -> G7 -> Uni
* Local Beam Search      :  28.85 min | Depart: 08:45 | Path: Home -> G1 -> G2 -> G3 -> G7 -> Uni
* Genetic Algorithm      :  28.85 min | Depart: 08:45 | Path: Home -> G1 -> G2 -> G3 -> G7 -> Uni
================================================================
```

Empirical Comparison of Local Search Algorithms

Scenario	Algorithm	Best Time f(x*)	Depart	Optimal Path
Rush Hour (Trap)	Hill Climbing	**41.77**	08:45	Home -> B1 -> B2 -> H2 -> Uni
	Simulated Annealing	48.29	08:45	Home -> H1 -> B1 -> B2 -> Uni
	Local Beam Search	**41.77**	08:45	Home -> B1 -> B2 -> H2 -> Uni
	Genetic Algorithm	45.01	07:45	Home -> B1 -> B2 -> Uni
Dual Peak (Multi-Modal)	Hill Climbing	**31.69**	08:45	Home -> C1 -> C2 -> T2 -> Uni
	Simulated Annealing	38.57	08:45	Home -> N1 -> B1 -> C2 -> T2 -> Uni
	Local Beam Search	**31.69**	08:45	Home -> C1 -> C2 -> T2 -> Uni
	Genetic Algorithm	**31.69**	08:45	Home -> C1 -> C2 -> T2 -> Uni
Cascading Gridlock (Trap)	Hill Climbing	**31.02**	08:45	Home -> C1 -> C2 -> T2 -> Uni
	Simulated Annealing	32.00	08:45	Home -> N1 -> B1 -> C2 -> B2 -> Uni
	Local Beam Search	**31.02**	08:45	Home -> C1 -> C2 -> T2 -> Uni
	Genetic Algorithm	**31.02**	08:45	Home -> C1 -> C2 -> T2 -> Uni
Crossover Showcase (GA Test)	Hill Climbing	32.00	07:45	Home -> N1 -> B1 -> C2 -> B2 -> Uni
	Simulated Annealing	32.00	07:50	Home -> N1 -> B1 -> C2 -> T2 -> Uni
	Local Beam Search	**31.01**	08:45	Home -> C1 -> C2 -> B2 -> Uni
	Genetic Algorithm	32.00	07:45	Home -> N1 -> B1 -> C2 -> B2 -> Uni
Flat Plateau (Plateau Test)	Hill Climbing	**28.85**	08:45	Home -> G1 -> G2 -> G3 -> G7 -> Uni
	Simulated Annealing	**28.85**	08:45	Home -> G1 -> G2 -> G3 -> G7 -> Uni
	Local Beam Search	**28.85**	08:45	Home -> G1 -> G2 -> G3 -> G7 -> Uni
	Genetic Algorithm	**28.85**	08:45	Home -> G1 -> G2 -> G3 -> G7 -> Uni

3 – تصویر نمودار همگرایی و تحلیل مقایسه ای الگوریتم بر روی دشت ، قله و دره محلی :





- الگوریتم Hill Climbing : ابتدا با شیب زیاد کاهش می یابد اما پرش های عجیب در نقاطی دارد که به دلیل Random Restart و گیر کردن در یک دره محلی ، در نهایت در یک دره محلی گیر کرده و صاف می شود.
  1. دشت : با استفاده از حرکات افقی روی آن راه می رود تا شاید راهی به پایین پیدا کند
  2. قله : هرگز به آنجا نمی رود
  3. دره محلی : در آن گیر می کند و متوقف می شود
- الگوریتم Simulated Annealing : بیشترین نوسانات را دارد .
  1. دشت : به راحتی عبور می کند
  2. قله : در دمای بالا ، موقتا قبول می کند تا شاید یک راه بهتری به سمت دره اصلی پیدا کند
  3. دره محلی : با پذیرش احتمالی مسیر بدتر از آن خارج می شود.

- الگوریتم Beam search : به دلیل نگه داشتن k مسیر موازی ، یک شیب نزولی یکنواخت و بدون نوسان دارد .
  1. دشت : چون k مسیر موازی دارد ، احتمالا حداقل یکی از مسیر ها راه خروج از دشت را پیدا می کند.
  2. قله : در صورت یک مسیر اشتباه به قله ، کل مسیر ها درگیر نمی شوند
  3. دره محلی : اگر یک یا چند مسیر در دره گیر کنند ، مسیرهای موازی دیگر شانس پیدا کردن راه بهتر دارند و الگوریتم را نجات می دهند.

- الگوریتم Genetic With Elitism : در اینجا چون نخبه ها را حفظ می کنیم ، جواب خیلی سریع به سمت بهینه شدن و خط صاف شدن می رود و جواب بدتر قبول نمی کند .
  1. دشت : با استفاده از Mutation و Crossover تنوع ایجاد می کند و از دشت عبور کرده.
  2. قله : به سمت قله های بد نمی رود چون Elitism دارد.
  3. دره محلی : با ترکیب والدین مختلف و ایجاد فرزند جدید ، از دره محلی فرار می کند تا شاید دره عمیق تر یا همان جواب بهینه تر پیدا کند .

پرسش ها :

سوالات پرسش ها به دلیل نداشتن شماره با الف و ب نام گذاری شده

1 – الف) بهینه سازی یک مسئله ترکیبیاتی در فضای گسسته است اما الگوریتم گرادیان کاهشی در فضای پیوسته است و برای انتخاب یال ، چیزی به نام شیب نداریم .

ب) بله ، اگر $C_t(e)$ ها باید مشتق پذیر باشند و همچنین محدب (یا Convex) باشد

---

2 – الف) چون هزینه ها وابسته به زمان است پس تصمیم گیره گره به گره باعث افتادن در تله های ترافیکی پایین دستی می شود پس باید کل مسیر و زمان یکجا بررسی شوند .

ب) الگوریتم های تدریجی در صورت آزاد بودن ساعت حرکت t_{start} باید برای تمام بازه های زمانی از ابتدا اجرا شوند که نا کار آمد هست .

---

3 – الف) مزیت Crossover این است که با ترکیب زیر مسیر های دو والد ، ساختار های جدید و برتر می سازد که هیچ کدام از والدین به تنهایی نداشتند اما beam search صرفا k جواب برتر را کپی می کند و ساختار ها را ترکیب نمی کند.

ب) بله ، ما در کد از یک مجموعه seen_path استفاده کردیم که از تکرار مسیرها جلوگیری کردیم .برای حفظ تنوع بیشتر می توانیم به جای حریصانه از انتخاب احتمالی استفاده کنیم.

---

4 – الف) در این شبکه ، ترافیک یکنواخت باعث می شود مسیرهای موازی زیادی زمان یکسان داشته باشد ، این وضعیت دشت ایجاد می کند که الگوریتم های حریصانه در آن متوقف می شوند.

ب) افزایش این ابر پارامتر زمان اجرا را کمی بیشتر می کند ، اما با جلوگیری از توقف زودهنگام روی دشت ، شانس الگوریتم برای پیدا کردن مسیر بهینه بیشتر می شود .

---

5 – الف) جستجو شبیه یک پیمایش تصادفی می شود و الگوریتم زمان زیادی را صرف پذیرش مسیر بد می کند و ممکن است در تکرار معین همگرا نشود .

ب) در حالت $\alpha = 0.5$ کاوش بسیار سطحی و کم است والگوریتم زودتر در تله محلی گیر می کند اما در $\alpha = 0.99$ کاوش بسیار عمیق تر و شانس یافتن جواب سراسری(Global) بالا می رود اما به تکرار بسیار زیاد و زمان بالا نیاز دارد .

---

6 – الف) اگر مجموعه گره مشترک ، خالی باشد کد ما وارد قسمت جایگزین می شود یعنی برداشتن مسیر والد اول با تغییر جزئی در زمان حرکت آن را به عنوان فرزند بر می گرداند.

```
if not common_nodes:
    new_time = parent1.departure_time + random.choice([-problem.t_step, problem.t_step])
    new_time = max(problem.t_min, min(problem.t_max, new_time))
    return State(path=parent1.path, departure_time=new_time)
```

ب) چون ممکن است دو نیمه هر کدام خوب باشند اما وقتی با هم ترکیب می شوند هماهنگی زمانی را بهم بزنند .

اگرچه از نظر فیزیکی جاده ها روی نقشه به هم وصل هستند و مسیر هیچ جای پرش ندارد اما به تله ترافیکی می خورد و زمان را بیش از حد بالا می برد .

---

7 – الف)

$$|x| \leq b^L * N_t$$

ب) افزایش طول مسیر باعث رشد نمایی تعداد حالت ها می شود ، در شبکه های واقعی از نظر عملی غیر ممکن است چون ممکن است تعداد مسیر ها از مرز میلیون یا حتی میلیارد فراتر برود(L=30)، به همین دلیل از الگوریتم های Local search با کاوش هوشمندانه(تابع هزینه(f(x)) در زمان کوتاه تری به جواب نزدیک به بهینه می رسیم .
